Incidental

Requirements Document

GitHub Repository:
https://github.com/SandwichKCR/intermed-software-des

Ethan Skidmore, Ollivya Philip, Ravon Henson, Viet Tran, Saideep Katragadda

Incident Response Management System

Fully Dressed Use Cases – Incident Declaration

Use Case UC1: Declare New Incident

Primary Actor:
Incident Commander / System Operator

Stakeholders and Interests:

- **Responders:** Need clear incident information to begin investigation quickly.
- **Operations Team:** Needs fast declaration of outages to coordinate response.
- **Stakeholders / Management:** Want accurate incident records for communication and postmortems.
- **Customers:** Want incidents to be resolved as quickly as possible.

Preconditions:

- The user is authenticated and has permission to declare incidents.
- The incident response management system is operational.

Success Guarantee:

- A new incident is successfully created and stored in the system with a title and description.
- The incident is visible to relevant responders and teams.

Main Success Scenario:

1. The actor selects the option to declare a new incident.
2. The system displays a form requesting incident information.
3. The actor enters the incident title.
4. The actor enters a description explaining the issue.
5. The actor submits the incident declaration.
6. The system validates the information.

7. The system creates and stores the new incident record.
8. The system confirms the incident has been successfully created.

Extensions:

- 4a. Title field is empty
    - The system prompts the actor to enter a valid title.
- 5a. Description is incomplete
    - The system requests additional details before submission.

Special Requirements:

- Incident creation must occur within seconds to allow rapid response coordination.

Technology and Data Variation List:

- Incident information may be entered via web interface, API, or command-line integration.

Use Case UC2: Assign Incident Severity

Primary Actor:
Incident Commander

Stakeholders and Interests:

- **Operations Team:** Needs correct severity classification to prioritize response.
- **Management:** Needs accurate severity to determine communication level.
- **Customers:** Critical incidents should be resolved faster.

Preconditions:

- An incident has been created in the system.

Success Guarantee:

- The incident is assigned a severity level (Low, Medium, Critical).

Main Success Scenario:

1. The actor opens an incident record.
2. The system displays incident details.
3. The actor selects a severity level.
4. The actor confirms the selection.
5. The system updates the incident severity.
6. The system records the severity change in the incident timeline.

Extensions:

- 3a. Actor selects an invalid severity level
    - The system displays an error message and requests a valid selection.

Special Requirements:

- Severity updates must be immediately visible to all responders.

Technology and Data Variation List:

- Severity levels include predefined categories: Low, Medium, Critical.

Use Case UC3: Categorize Incident Type

Primary Actor:
Incident Commander

Stakeholders and Interests:

- **Engineering Teams:** Need incident categorization to determine the appropriate expertise required.
- **Security Teams:** Need to identify security-related incidents quickly.
- **Operations Management:** Uses incident categories for analysis and reporting.

Preconditions:

- The incident has already been declared.

Success Guarantee:

- The incident is categorized according to type.

Main Success Scenario:

1. The actor views an existing incident.
2. The system displays incident classification options.
3. The actor selects an incident type.
4. The system records the selected category.
5. The system updates the incident metadata.

Extensions:

- 3a. Actor attempts to assign multiple conflicting categories
    - The system requests clarification or allows only one primary category.

Special Requirements:

- Categories must follow predefined types (Outage, Security, Performance, etc.).

Technology and Data Variation List:

- Incident categories may be expanded by system administrators.

Use Case UC4: Assign Initial Responder or Response Team

Primary Actor:
Incident Commander

Stakeholders and Interests:

- **Responders:** Need immediate notification when assigned to incidents.
- **Operations Team:** Needs clear ownership of incidents.
- **Management:** Needs accountability during incident response.

Preconditions:

- An incident has been declared.

- Responders or response teams exist in the system directory.

Success Guarantee:

- The incident is assigned to a responder or response team.

Main Success Scenario:

1. The actor opens the incident record.
2. The actor selects the option to assign a responder or team.
3. The system displays available responders or teams.
4. The actor selects the appropriate responder or team.
5. The system records the assignment.
6. The system notifies the assigned responders.

Extensions:

- 4a. Selected responder is unavailable
    - The system suggests alternative responders or teams.

Special Requirements:

- Assigned responders must receive immediate notification.

Technology and Data Variation List:

- Notifications may occur through system dashboards, email, or messaging integrations.

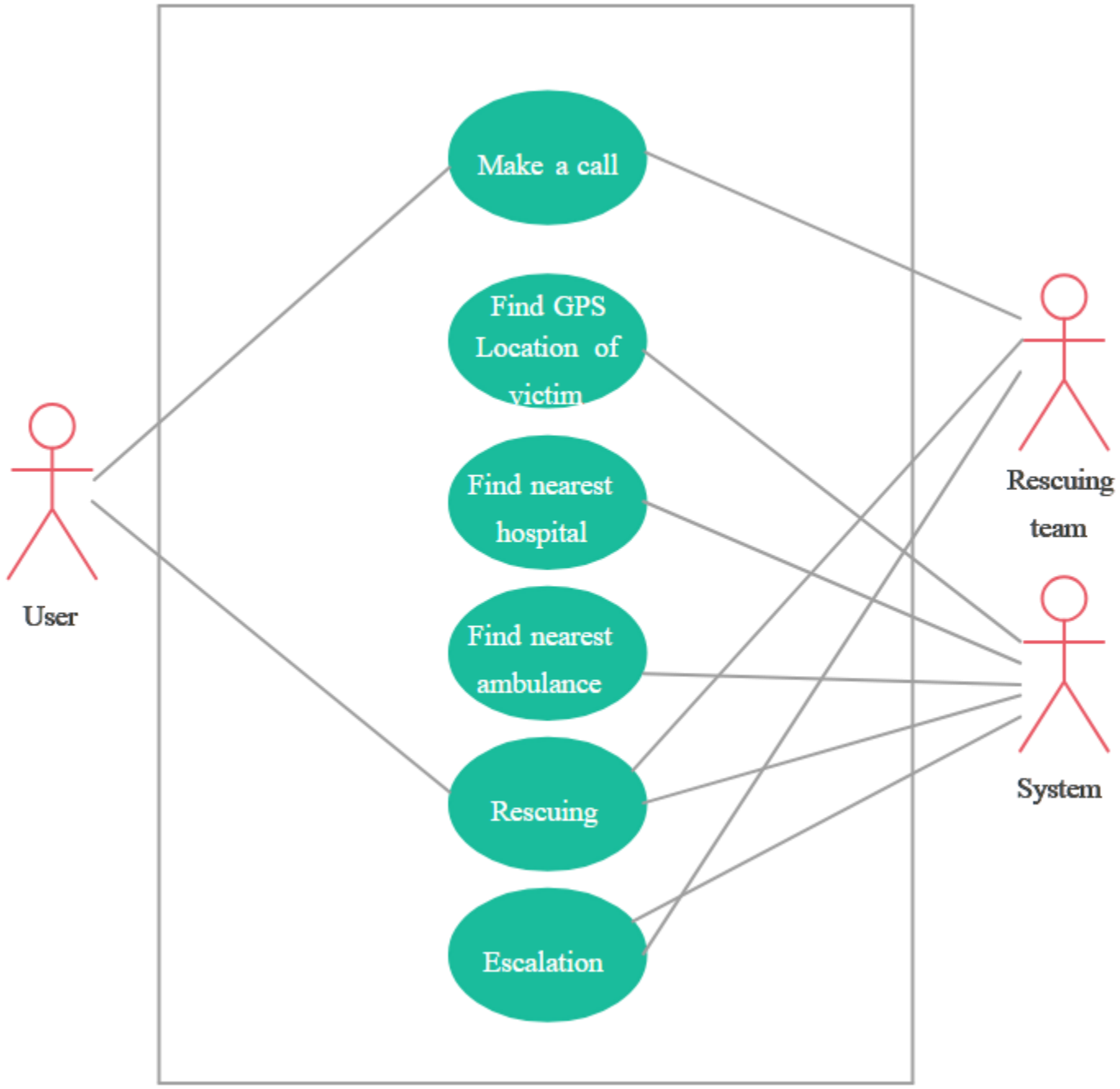
Supplementary Specifications (Non-Functional Requirements)

Reliable Incident Data Persistence

The system must reliably persist incident data without confirmed data loss, even during concurrent incident declarations. This ensures that incident records, responder assignments, and severity levels remain consistent and recoverable even when multiple users declare incidents simultaneously.

This non-functional requirement focuses on **data durability, system reliability, and concurrency handling** in the incident management platform.

Use Case Diagram:



Incidental Requirements — Status Feature (Ethan Skidmore)

Scope

This document captures functional and non-functional requirements for the **Status** component of *Incidental,* a real-time incident reporting and management system.

Fully dressed use cases (Status)

UC1: Status Creation

- **Primary Actor:** Affected Service
- **Stakeholders and Interests:**
    - Affected Service: Create and send status update for internal teams and to public
    - Affected Service User: Stay notified on status of service
- **Preconditions:**
    - Service registered with *Incidental*
    - User must be logged in and affiliated with service
    - User must be in status tab of incident report
- **Success Guarantee:**
    - Status is created fully, is saved to high-level account, and is ready to be sent, but is not yet sent
- **Main Success Scenario:**
    1. Status tab is displayed with current status or no status, "Create New Status" and "Load Status Preset" buttons
    2. User selects "Create New Status" button
    3. Status creation screen is displayed
    4. Editable fields for status name, notes, and other details
    5. "Revert" button and "Save" button are present on this screen
    6. User sets each field to desired values
    7. User clicks "Save" button, status is saved to service account, and incident status is updated on account-side
    8. Status edit window closes and returns user to status tab
- **Extensions:**

    2a. User selects "Load Status Preset" button
        1. List of saved status presets is displayed to user
        2. User selects a preset to load
        3. status is saved to service account, and incident status is updated on account-side
        4. Status edit window closes and returns user to status tab

    7a. User selects "Revert" button
        1. Changes to status are reverted as if no editing had taken place
        2. Status editing window is closed
- **Special Requirements:**
- **Technology and Data Variations:**
    - Statuses can be created on the spot or loaded from saved preset

UC2: Status Preset Management

- **Primary Actor:** Affected Service
- **Stakeholders and Interests:**
    - Affected Service: Be able to quickly update status without having to create an entire update every time
    - Affected Service User: Receive more timely status updates
- **Preconditions:**
    - Service registered with *Incidental*
    - User must be logged in and affiliated with service
- **Success Guarantee:**
    - Status preset is created that can be selected any time, without the need to create a status from scratch
- **Main Success Scenario:**
    1. User opens "Status Presets" tab from main menu
    2. User is presented with a current list of status presets, with options to delete, edit, and create new status presets
    3. User selects "create new status preset"
    4. A window opens with status field options, similar to creating an incident status
    5. "Cancel" and "Save Preset" buttons are on this screen
    6. User fills out fields and selects "Save Preset"
    7. New status window closes, and new status preset is added to list
- **Extensions:**

    3a. User selects a preset to delete
        1. Preset is deleted from the list
        2. User is returned control

    3b. User selects a preset to edit
        1. A status editing window opens with fields already filled out from selected preset
        2. Return to step 5

    6a. User selects "Cancel"
        1. Preset is deleted
        2. User returns to list

- **Special Requirements:**
- **Technology and Data Variations:**
    - User can edit existing presets or create new ones

UC3: Status Publishing

- **Primary Actor:** Affected Service
- **Stakeholders and Interests:**
    - Affected Service: Coordinate teams with specific statuses without notifying public of secret information
- **Preconditions:**
    - Service registered with *Incidental*

    - User must be logged in and affiliated with service
    - User must be in status tab of incident report
    - Status must have been created previously (either just now, or saved as preset)
- **Success Guarantee:**
    - Status is displayed selectively on *Incidental* page to either team, public, or both
    - Users affiliated with service can now view and edit status
- **Main Success Scenario:**
    1. Next to loaded or created status is a button to publish status to public, or publish to internal team members
    2. User presses "Publish to public" button
    3. Status is updated on public *Incidental* page, as well as on internal page for team members and affiliates
    4. User is notified of successful publish
- **Extensions:**
    2a. User selects publish to internal team members
        1. All service-affiliated users, including those who are not high-level, receive status update and can view status
        2. Return to step 4
- **Special Requirements:**
    - Status must reach internal team within 10 seconds, and other users within 5 minutes
- **Technology and Data Variations:**
    - None

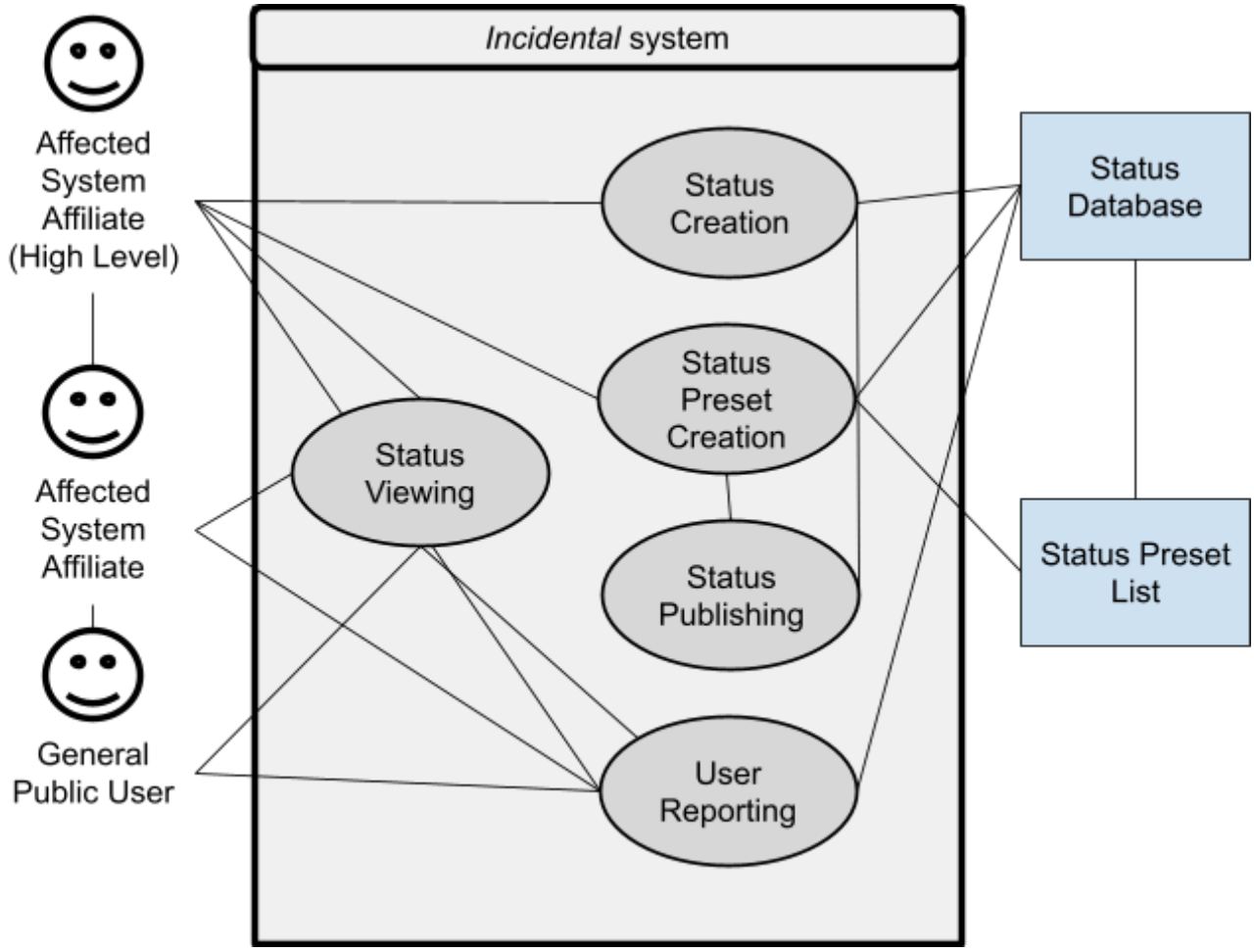
UC4: Stakeholder Reports Influence Status

- **Primary Actor:** Users of Affected Service
- **Stakeholders and Interests:**
    - Affected Service User: Can report incidents and update status as a stakeholder, also can tell if issue is on their end or is affecting many users
    - Affected Service: Can get quick reports from users, as well as assess breadth of incident
- **Preconditions:**
    - Service registered with *Incidental*
    - User must be on page for service
    - User reporting feature must be toggled on by service
- **Success Guarantee:**
    - User report will have been sent to affected service
    - Report counter on status will be updated
- **Main Success Scenario:**
    1. User accesses service page, "Report Issue" button is displayed
    2. Optional report description text box is available to user
    3. User enters report information and presses report button
    4. Report is sent to service, and internal report counter is updated
    5. If incident and status are currently active, status will include report counter displayed to target group

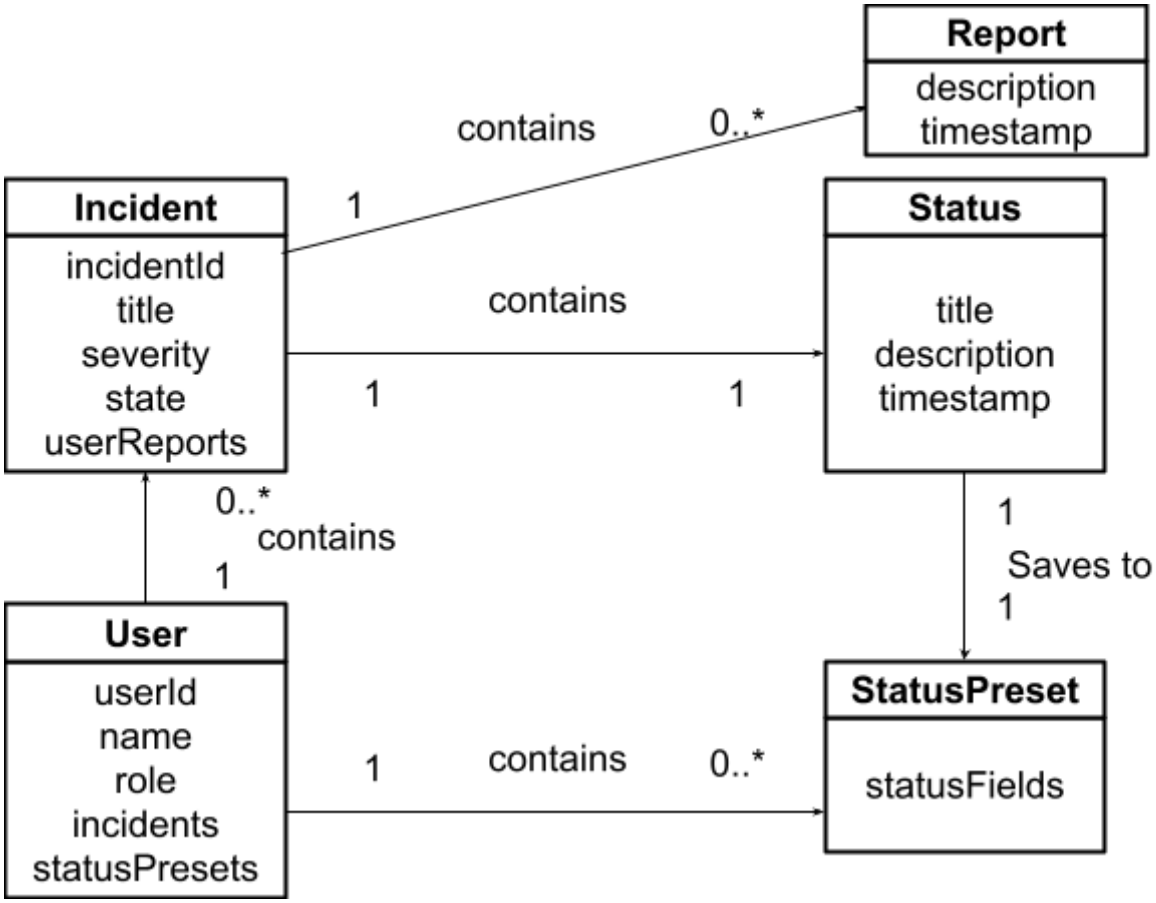
6. Notification of successful report will be sent to user

- **Extensions:**
- **Special Requirements:**
- **Technology and Data Variations:**
    - User can send empty report
    - User or service affiliates can send reports
    - User does not need to have an account

Use case diagrams



Conceptual class diagrams



Supplementary Specifications

Status must reach the internal team within 10 seconds, and other users within 5 minutes - any status updates sent must reach the internal team quickly (< 10 seconds) in order for them to be able to quickly respond to incidents and status updates. The general public can have a much larger delay, so up to 5 minutes is allowable.

Incidental Requirements - Incident Escalation & Severity Management (Viet Tran)

1. Scope:

This document captures functional and non-functional requirements for the **Incident Escalation and Severity Management** component of *Incidental*, a real-time incident reporting and management system.

2. Fully Dressed Use Cases (Escalation & Severity Management)

Use Case UC-ESM-01: Assign Responder to Incident

Primary Actor: Incident Commander

Supporting Actors: System

Stakeholders and Interests

- Incident Commander: Wants to quickly assign engineers or teams to an active incident.
- Responders: Want to be notified promptly when assigned so they can begin work.
- Managers: Want accurate records of who is responsible for an incident at any given time.

Preconditions

- An incident exists and is active.
- User is authenticated and authorized to assign responders.

Success Guarantee (Postconditions)

- Selected responders are linked to the incident record.
- Each assigned responder receives a notification.
- The assignment is logged in assignment history with a timestamp.

Main Success Scenario

1. Actor navigates to the active incident.
2. Actor selects "Assign Responder."
3. System displays available engineers, teams, and on-call groups.
4. Actor selects one or more responders.
5. System validates and assigns the responders to the incident.
6. System notifies assigned responders and updates the incident view for all active users.
7. System logs the action to assignment history.

Extensions

- 4a. Actor selects an on-call group:
    1. System resolves the group to the currently active on-call engineer.
    2. Resume from step 5.
- 5a. Responder is already assigned:
    1. System alerts the actor and skips the duplicate.

Special Requirements

- Assignment updates must propagate to all active users within 2 seconds.

Technology and Data Variations

- Responders may be individuals, teams, or on-call groups from a scheduling integration.

Frequency of Occurrence: High during active incidents.

Use Case UC-ESM-02: Reassign or Remove Responder

Primary Actor: Incident Commander

Supporting Actors: System

Stakeholders and Interests

- Incident Commander: Wants to update the roster during an incident due to shift changes or escalations.
- Managers/Auditors: Want all roster changes logged for accountability.

Preconditions

- An incident exists and is active.
- At least one responder is currently assigned.
- User is authorized to manage responders.

Success Guarantee (Postconditions)

- The incident's responder list reflects the changes made.
- Affected responders are notified.
- The change is logged in assignment history with a timestamp.

Main Success Scenario

1. Actor opens the active incident and navigates to the Responders panel.
2. Actor selects a responder and chooses "Reassign" or "Remove."
3. (For Reassign) System displays available responders; actor selects a replacement.

4. System updates the responder list and logs the change.
5. System notifies affected responders and updates the view for all active users.

Extensions

- 2a. Actor removes a responder with no replacement:
    1. System removes the responder and warns the actor if the incident now has zero assigned responders.
- 3a. Selected replacement is unavailable:
    1. System rejects the change and retains the original responder.

Special Requirements

- Roster updates must propagate to all active users within 2 seconds.

Technology and Data Variations

- Notifications may be delivered via email, Slack, or configured alerting integration.

Frequency of Occurrence: Medium/High during active incidents.

Use Case UC-ESM-03: View Current Responders

Primary Actor: User (Responder/Engineer/Stakeholder with access)

Supporting Actors: System

Stakeholders and Interests

- Responders: Want to know who else is working on the incident for coordination.
- Stakeholders: Want to know who is responsible for resolving the incident.

Preconditions

- An incident exists.
- User has permission to view the incident.

Success Guarantee (Postconditions)

- User sees the current list of assigned responders with names, roles, and assignment times.
- No system state is changed.

Main Success Scenario

1. User opens the incident detail view.
2. System retrieves and displays the current responder list.
3. Each entry shows the responder's name, role/team, and assignment timestamp.
4. The list refreshes automatically when assignment changes occur.

Extensions

- 2a. No responders are assigned:
    1. System displays a message indicating no responders are currently assigned.
- 2b. Data fails to load:
    1. System displays an error and offers a retry option.

Special Requirements

- The responder list must reflect changes within 2 seconds without a manual page reload.

Technology and Data Variations

- Sensitive contact details may be hidden for stakeholder roles per access control configuration.

Frequency of Occurrence: High during active incidents.

Use Case UC-ESM-04: View Assignment History

Primary Actor: Incident Commander, Auditor

Supporting Actors: System

Stakeholders and Interests

- Incident Commander: Wants a full record of assignment changes for situational awareness.
- Auditors/Managers: Want an immutable log for accountability and compliance.
- Postmortem Owners: Want complete history to support accurate incident reports.

Preconditions

- An incident exists (active or resolved).
- User has read access to the incident's history.

Success Guarantee (Postconditions)

- User sees a chronological log of all assignment, reassignment, and removal events.
- Each entry shows event type, responder name, acting user, and timestamp.

- No system state is changed.

Main Success Scenario

1. User opens the incident detail view and selects the "Assignment History" tab.
2. System retrieves and displays all assignment events in chronological order.
3. User can filter by date range or event type.
4. User optionally exports the history as a report.

Extensions

- 2a. No history exists:
    1. System displays a message stating no assignment history is available.
- 2b. Data fails to load:
    1. System displays an error and offers a retry option.

Special Requirements

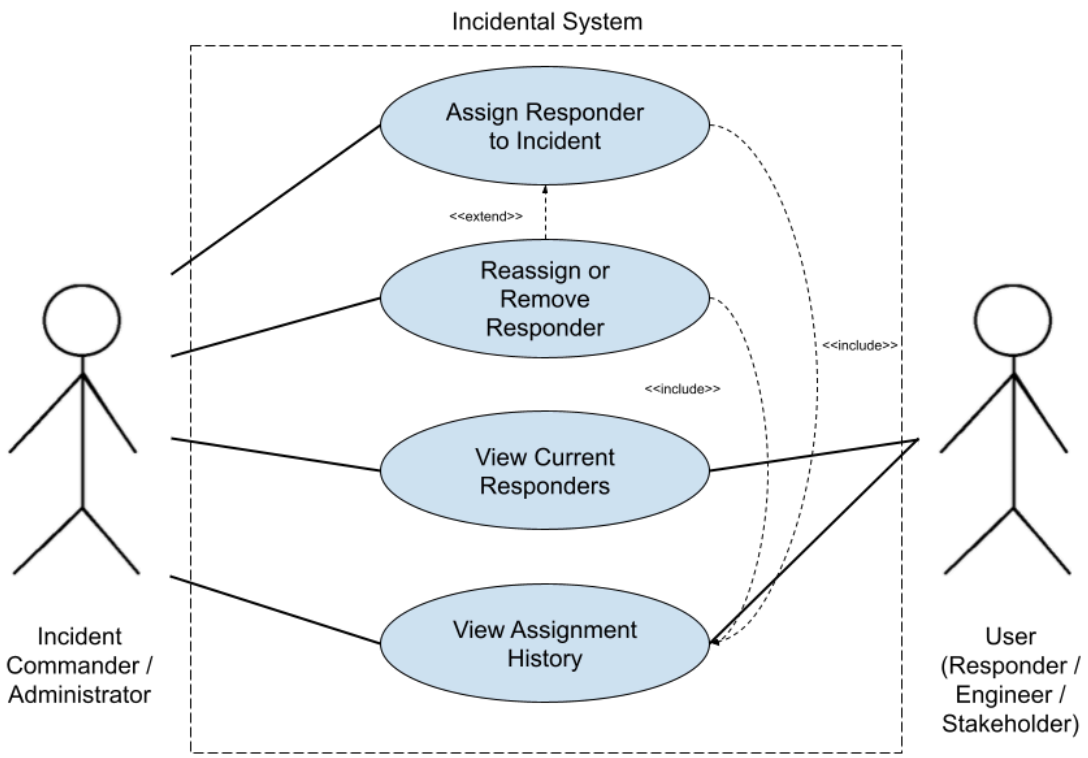
- Assignment history records must be immutable (append-only) once created.

Technology and Data Variations

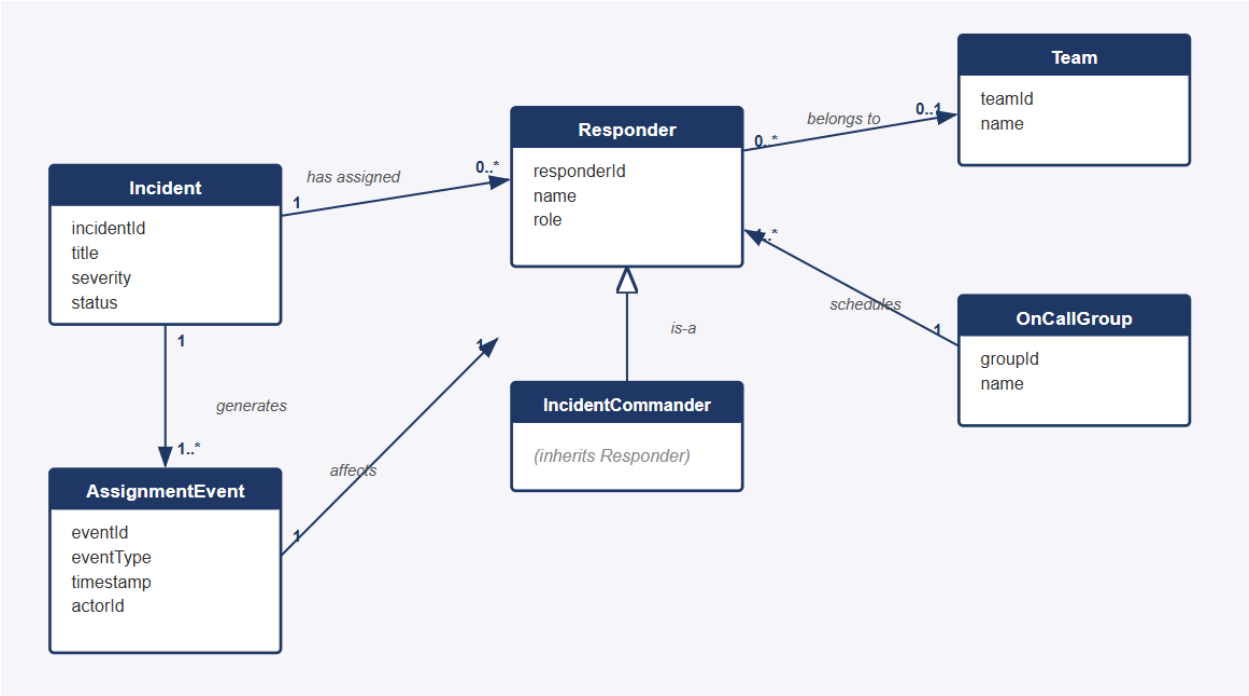
- History may be exported as CSV or PDF for postmortem or compliance use.

Frequency of Occurrence: Medium.

3. Use Case Diagram (Escalation & Severity Management)



4. Conceptual Class Diagram (Escalation & Severity Management Domain Model)



5. Supplementary Specifications (Non-Functional Requirements)

NFR-ESM-1 (Performance / Real-Time Propagation): Responder assignment updates must be reflected to all active users within 2 seconds under normal operating conditions.

Explanation: This applies to all assignment, reassignment, and removal events. Compliance is measured from when the change is accepted by the system to when it becomes visible to other active users. The system must use a real-time push mechanism such as WebSockets or Server-Sent Events to achieve this, ensuring all participants have a consistent view of the responder roster at all times.

Incidental Requirements — Timeline Feature (Ravon Henson)

1 Scope

This document captures functional and non-functional requirements for the **Timeline** component of *Incidental*, a real-time incident reporting and management system.

2 Fully Dressed Use Cases (Timeline)

Use Case UC-TL-1: Record System-Generated Timeline Event

Primary Actor: System (Incidental)
Supporting Actors: Integrations/Monitoring (optional), Active users (viewers)

Stakeholders and Interests

- **Responders:** want an accurate, chronological history of what happened.
- **Managers:** want an auditable timeline for review and accountability.
- **Stakeholders:** want trustworthy incident progress information.

Preconditions

- An incident exists and is active.

Success Guarantee (Postconditions)

- A new timeline entry is stored with a server-generated timestamp.
- Timeline order remains strictly chronological (no manual reordering).

Main Success Scenario

1. A system event occurs (e.g., incident declared, responder assigned, status updated).
2. The system creates a timeline entry describing the event.
3. The system assigns a server-generated timestamp to the entry.
4. The system stores the entry and appends it to the incident timeline.
5. The system displays the updated timeline to active users.

Extensions

- **2a. High-frequency events occur simultaneously**
  1. The system uses a single authoritative timestamp source.
  2. The system orders events by timestamp (and a stable tie-breaker when needed).
- **4a. Storage failure**
  1. The system signals an error and retries persistence.

2. If retries fail, the system flags timeline persistence as degraded and alerts authorized users.

Special Requirements

- Users cannot manually reorder entries.

Technology and Data Variations

- Events may originate from user actions, integrations, or internal services.

Frequency of Occurrence: High during active incidents.

Use Case UC-TL-2: Add Manual Timeline Entry

Primary Actor: Authorized User (Responder/Engineer)
Supporting Actors: System

Stakeholders and Interests

- **Responders:** want to record actions taken, observations, and decisions for coordination.
- **Postmortem Owner:** wants complete records for later reporting.

Preconditions

- An incident exists.
- User is authorized to add timeline entries.

Success Guarantee (Postconditions)

- The entry is stored and visible in the timeline.
- The entry has a server-generated timestamp.

Main Success Scenario

1. User opens the incident timeline.
2. User selects "Add Timeline Entry."
3. System prompts for entry details (event type and description).
4. User enters description and selects an event type (e.g., decision, mitigation, responder action, system event).
5. System assigns a server-generated timestamp.
6. System stores the entry and displays it in chronological order.

Extensions

- **4a. Missing required fields**
  1. System prompts user to complete missing information.
  2. User revises and resubmits.
- **2a/4b. User not authorized**
  1. System rejects the request and displays an authorization error.

Special Requirements

- Users cannot set or edit timestamps.
- Users cannot reorder entries.

Technology and Data Variations

- Event type values may be selected from a predefined list (e.g., decision, mitigation, responder action, system event).

Frequency of Occurrence: High during active incidents.

Use Case UC-TL-3: Edit Timeline Entry with Revision History

Primary Actor: Authorized User (Responder/Engineer)
Supporting Actors: System

Stakeholders and Interests

- **Responders:** want to correct inaccurate or incomplete entries.
- **Managers/Auditors:** want edit history preserved for accountability.

Preconditions

- An incident exists.
- The timeline entry exists.
- User is authorized to edit timeline entries.

Success Guarantee (Postconditions)

- Updated entry content is saved.
- A revision record is created and previous versions remain accessible.
- The original event timestamp remains unchanged.

Main Success Scenario

1. User opens the incident timeline.
2. User selects an existing timeline entry.
3. User selects "Edit."
4. System displays the current entry content.
5. User modifies the content and submits changes.
6. System saves the updated entry.
7. System stores the prior version in revision history.
8. System indicates the entry has been edited and provides access to revisions.

Extensions

- **5a. Concurrent edits (conflict)**
  1. System detects a conflict.
  2. System prompts the user to refresh and reapply changes (or merge, if supported).
- **2a/5b. User not authorized**
  1. System rejects the edit and displays an authorization error.

Special Requirements

- Revision history must preserve previous versions.
- Users cannot modify the original timestamp of a timeline entry.

Technology and Data Variations

- Revision history may be displayed as a list of prior versions for a selected entry.

Frequency of Occurrence: Medium.

Use Case UC-TL-4: Filter Timeline by Event Type

Primary Actor: User (Responder/Engineer/Stakeholder with access)
Supporting Actors: System

Stakeholders and Interests

- **Responders:** want to quickly find relevant actions and decisions.
- **Stakeholders:** want a simplified view of relevant events.

Preconditions

- An incident exists.
- User has permission to view the timeline.

Success Guarantee (Postconditions)

- Timeline view updates to show only entries matching selected event type(s).
- Underlying stored events and ordering are unchanged.

Main Success Scenario

1. User opens the incident timeline.
2. System displays the full timeline in chronological order.
3. User selects one or more event type filters.
4. System updates the view to show only matching entries.
5. User clears filters to return to the full timeline.

Extensions

- **3a. No entries match**
    1. System shows an empty-state message.
    2. System allows the user to clear filters to return to full view.

Special Requirements

- Filtering must not alter underlying stored ordering.

Technology and Data Variations

- Filters may be single-select or multi-select depending on UI configuration.

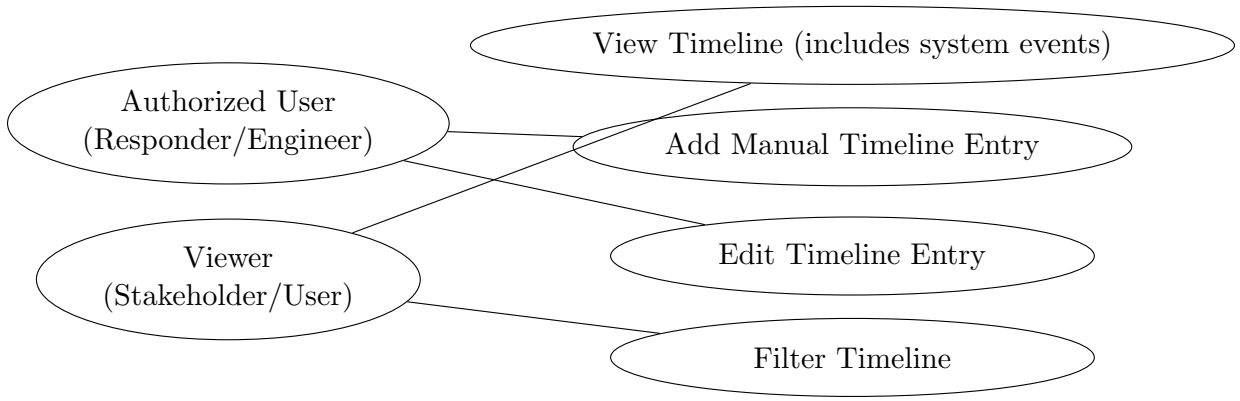
Frequency of Occurrence: Medium/High.

3 Supplementary Specifications (Non-Functional Requirements)

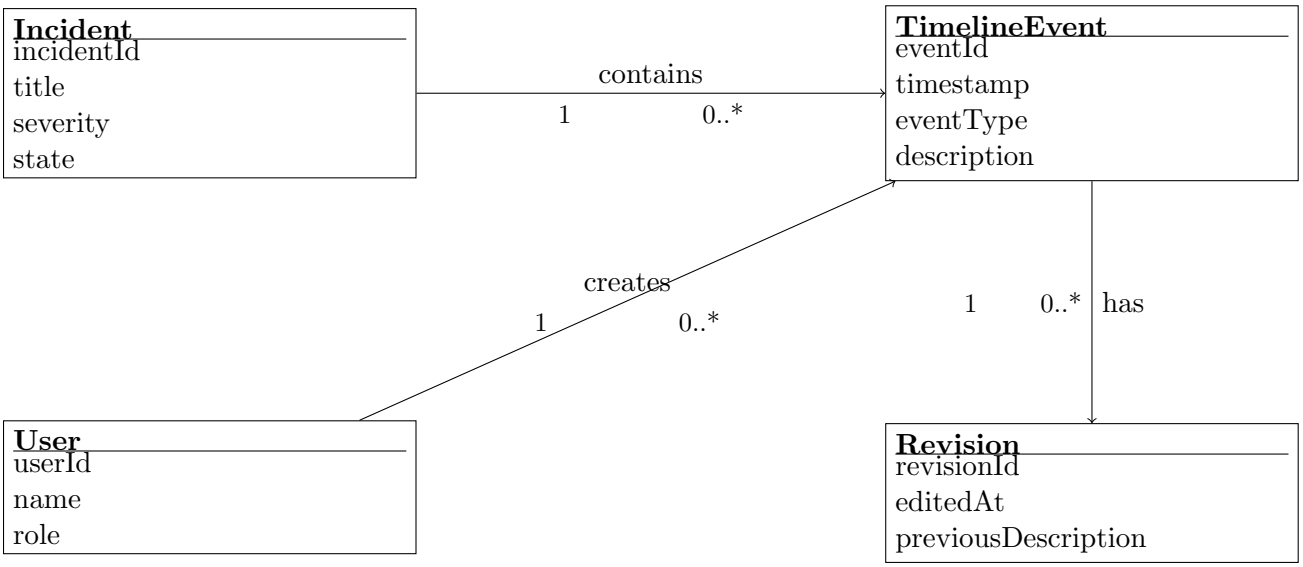
NFR-TL-1 (Performance / Real-time Collaboration): Timeline updates shall be reflected to all active users within **2 seconds** under normal operating conditions.

Explanation: This requirement applies to system-generated events, manual entries, edits, and timeline refreshes. Compliance is measured by the time from when an event is accepted by the system to when it becomes visible to other active users.

4 Use Case Diagram (Timeline)



5 Conceptual Class Diagram (Timeline Domain Model)



Incidental Requirements — Postmortem Generation Feature

1 Scope

This document captures functional and non-functional requirements for the **Postmortem Generation** component of *Incidental*, a real-time incident reporting and management system. The postmortem feature supports automatic draft generation at incident closure, structured completion requirements, review/approval prior to publication, and creation/tracking of follow-up action items.

2 Fully Dressed Use Cases (Postmortem Generation)

Use Case UC-PM-1: Automatically Generate Postmortem Draft on Incident Closure

Primary Actor: System (Incidental)
Supporting Actors: Authorized User (Responder/Engineer), Approver (Manager/Lead)

Stakeholders and Interests

- **Responders:** want an automatically created draft to reduce manual reporting effort.
- **Managers/Auditors:** want a consistent artifact generated for accountability and learning.
- **Stakeholders:** want a trustworthy summary of what happened and what was done.

Preconditions

- An incident exists.
- The incident is being closed.
- Incident data (timeline entries, status updates, participants, impact indicators) is available.

Success Guarantee (Postconditions)

- A postmortem **draft** is created and linked to the incident.
- The draft is populated with available incident data.

Main Success Scenario

1. The incident is closed.
2. The system gathers recorded incident data associated with the incident.
3. The system generates a postmortem draft using a standard template.
4. The system links the draft to the incident and makes it visible to authorized users.

Extensions

- **2a. Some incident data is missing**
    1. The system generates the draft using all available data.

    2. The system flags missing sections as incomplete for later completion.
- **3a. Draft generation fails**
    1. The system reports an error and retries postmortem draft creation.
    2. If retries fail, the system marks postmortem generation as degraded and alerts authorized users.

Special Requirements

- The system must not publish the postmortem automatically upon draft creation.

Technology and Data Variations

- Drafts may be generated from a configurable template (team-specific sections and required fields).

Frequency of Occurrence: Medium (once per closed incident).

Use Case UC-PM-2: Finalize Postmortem with Required Structured Sections

Primary Actor: Authorized User (Postmortem Owner / Responder)
Supporting Actors: System

Stakeholders and Interests

- **Postmortem Owner:** wants clear guidance on required content and completion.
- **Engineering Team:** wants consistent, complete postmortems for learning and prevention.

Preconditions

- A postmortem draft exists for the incident.
- User is authorized to edit the postmortem.

Success Guarantee (Postconditions)

- The postmortem is marked **finalized** only if all required sections are complete.

Main Success Scenario

1. User opens the postmortem draft.
2. System displays structured sections: root cause, impact, resolution, corrective actions.
3. User completes the required sections.
4. User selects "Finalize Postmortem."
5. System validates required sections are complete.
6. System marks the postmortem as finalized and ready for review/approval.

Extensions

- **5a. Required sections incomplete**
    1. System rejects finalization.
    2. System highlights missing required sections and prompts user to complete them.

Special Requirements

- The system must enforce completion of required structured sections before finalization.

Technology and Data Variations

- Required sections may vary by configured template; the system enforces configured required fields.

Frequency of Occurrence: Medium.

Use Case UC-PM-3: Review, Approve, and Publish Postmortem

Primary Actor: Approver (Manager/Lead)
Supporting Actors: Postmortem Owner, System

Stakeholders and Interests

- **Approver:** wants control to ensure accuracy and appropriate disclosure before publication.
- **Postmortem Owner:** wants a clear approval workflow.
- **Stakeholders:** want a reliable final report.

Preconditions

- A postmortem exists and is finalized (UC-PM-2).
- Approver is authorized to approve and publish postmortems.

Success Guarantee (Postconditions)

- The postmortem is marked approved.
- The postmortem is published to the intended audience.

Main Success Scenario

1. Approver opens the finalized postmortem.
2. Approver reviews the content.
3. Approver selects "Approve."
4. System records approval and marks the postmortem as approved.
5. Approver selects "Publish."
6. System publishes the postmortem.

Extensions

- **3a. Approver requests changes**
  1. Approver rejects approval and provides feedback.
  2. System marks the postmortem as "Needs Revision" and notifies the postmortem owner.
- **5a. Publication restricted**
  1. System enforces publication visibility controls (e.g., team-only, internal-only) based on configuration.

Special Requirements

- Approval must record approver identity and approval timestamp.
- Publication must not occur without approval.

Technology and Data Variations

- Publication target may vary by configuration (internal wiki, incident portal, export).

Frequency of Occurrence: Medium.

Use Case UC-PM-4: Create and Track Follow-Up Action Items

Primary Actor: Authorized User (Postmortem Owner / Responder / Manager)
Supporting Actors: Assignee(s), System

Stakeholders and Interests

- **Engineering Team:** wants corrective actions tracked to completion to prevent recurrence.
- **Managers:** want visibility into progress and accountability.
- **Assignees:** want clear ownership and status tracking.

Preconditions

- A postmortem exists (draft or finalized).
- User is authorized to create action items.

Success Guarantee (Postconditions)

- Action items are created, linked to the postmortem, and trackable by status.

Main Success Scenario

1. User opens the postmortem.
2. User selects "Create Action Item."
3. System prompts for action item details (description, assignee, due date if used).
4. User submits the action item.
5. System links the action item to the postmortem and sets status to "Open."
6. Authorized users update action item status as work progresses.
7. System displays action item status in the postmortem view.

Extensions

- **3a. Assignee not provided**
  1. System allows creation with an unassigned owner or prompts for assignment based on configuration.
- **6a. Unauthorized status update**
  1. System rejects the update and displays an authorization error.

Special Requirements

- Action items must remain linked to the postmortem and incident for traceability.

Technology and Data Variations

- Status values may be configured (e.g., Open, In Progress, Complete).

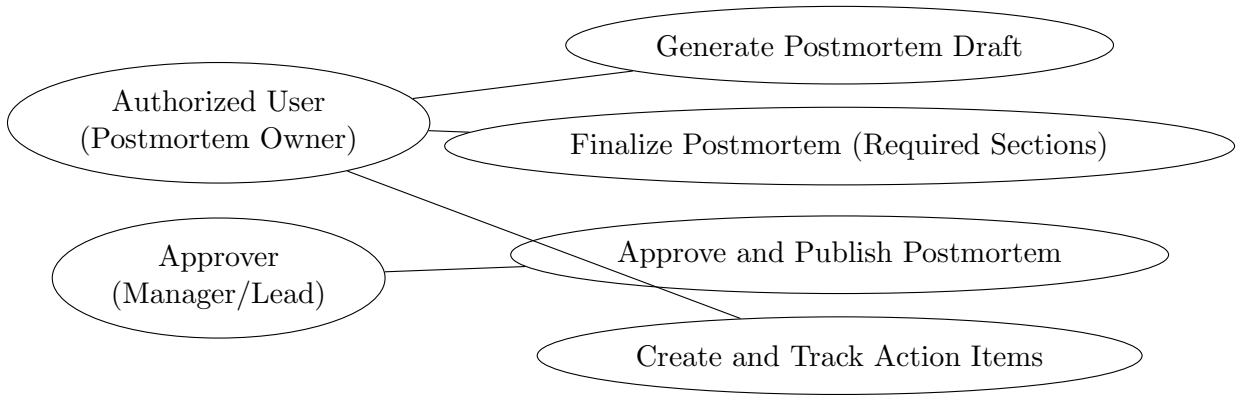
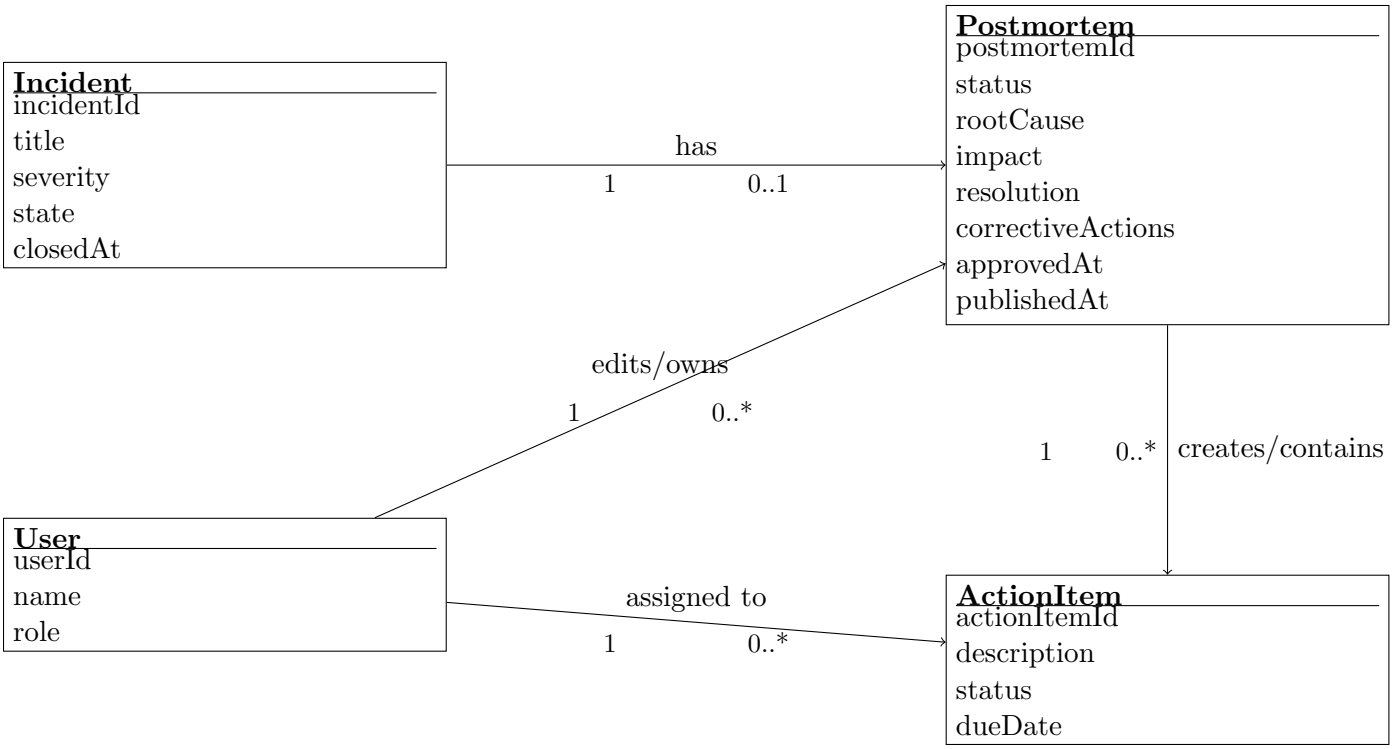
Frequency of Occurrence: Medium/High (multiple action items per postmortem).

3 Supplementary Specifications (Non-Functional Requirements)

NFR-PM-1 (Automation / Usability): Postmortem generation shall be automated and require minimal manual effort.

Explanation: Upon incident closure, the system automatically generates a draft populated with recorded incident data. Users should primarily complete structured sections and perform review/approval rather than manually assembling a report.

4 Use Case Diagram (Postmortem Generation)



5 Conceptual Class Diagram (Postmortem Generation Domain Model)